

**PROYECTO FINAL: SISTEMA DE DESCARGA
DE AGUA
(PARA LA CONAGUA)**

ANA PAMELA TINTORI KRIEG

FUNDAMENTOS DE PROGRAMACIÓN

INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un Sistema de Descarga de Agua realizado mediante Python. El programa fue desarrollado progresivamente durante el curso y en esta versión final se integraron las funciones realizadas en etapas anteriores junto con nuevas herramientas para el manejo de archivos, control de errores, estructuras de datos y depuración.

El sistema permite registrar información relacionada con empresas que realizan descargas de agua, consultar permisos, registrar el volumen de agua, identificar sustancias químicas y conservar la información mediante diferentes archivos de texto.

Para desarrollar el programa se utilizaron principalmente conceptos fundamentales de programación como variables, condicionales, ciclos, funciones, listas, matrices, tuplas, manejo de archivos y bloques try-except.

PROBLEMÁTICA

En un proceso de descarga de agua es necesario mantener un registro de las empresas que realizan las operaciones, sus permisos, el volumen de agua descargado y la posible presencia de sustancias químicas.

Si esta información se mantiene únicamente de forma temporal durante la ejecución de un programa, los datos se pierden al finalizarlo. Por esta razón, es necesario contar con un mecanismo que permita registrar y posteriormente consultar la información.

El proyecto plantea una solución académica mediante Python en la que los datos principales de las operaciones se almacenan dentro de archivos de texto.

OBJETIVO

El objetivo del proyecto es desarrollar un programa en Python que permita registrar y consultar información relacionada con descargas de agua mediante un menú de opciones.

El sistema debe permitir identificar al usuario y a la empresa, registrar la fecha de operación, validar información relacionada con los permisos, identificar sustancias químicas, registrar descargas, consultar información almacenada y utilizar archivos de texto para conservar los datos después de finalizar la ejecución del programa.

PARTE 1. IDENTIFICACIÓN DEL USUARIO Y BIENVENIDA

El programa comienza mediante la función `pant_inicio()`. Esta función solicita el nombre o nickname del usuario y posteriormente solicita el nombre de la empresa.

Después de capturar estos datos, el sistema muestra un mensaje de bienvenida personalizado utilizando el nombre del usuario y de la empresa.

Posteriormente se ejecuta la función `pant_carga(usuario)`, encargada de mostrar la pantalla de carga del sistema.

La pantalla de carga utiliza un ciclo `for` con tres iteraciones. En cada iteración se utiliza `time.sleep(1)`, produciendo una pausa aproximada de un segundo.

Debido a que el ciclo realiza tres iteraciones, la pantalla de carga tiene una duración aproximada de tres segundos, cumpliendo con el máximo de cinco segundos solicitado.

```
SISTEMA DE DESCARGA DE AGUA
Bienvenido/a Pamela . El sistema está listo para la empresa Demo
SISTEMA DE DESCARGA DE AGUA
Iniciando sistema...
Cargando.
Cargando..
Cargando...
Sistema Listo.
```

PARTE 2. CAPTURA DE FECHA

El sistema solicita la fecha correspondiente a la operación mediante la función `capturar_fecha()`.

El usuario debe introducir por separado el día, mes y año. Los valores ingresados son convertidos a datos numéricos mediante `int()`.

Posteriormente los tres valores se almacenan dentro de una tupla utilizando la estructura:

Fecha = día, mes, año

La función `mostrar_fecha()` recibe posteriormente la tupla y muestra sus tres elementos en pantalla.

También se utiliza `try-except` para capturar un `ValueError` en caso de que el usuario escriba texto u otro valor que no pueda convertirse a número.

```
Ingrese la fecha de la descarga:
Fecha de la descarga: 25 / 09 / 2026
```

PARTE 3. MENÚ PRINCIPAL

El sistema cuenta con un menú principal que permite acceder a las diferentes funciones disponibles.

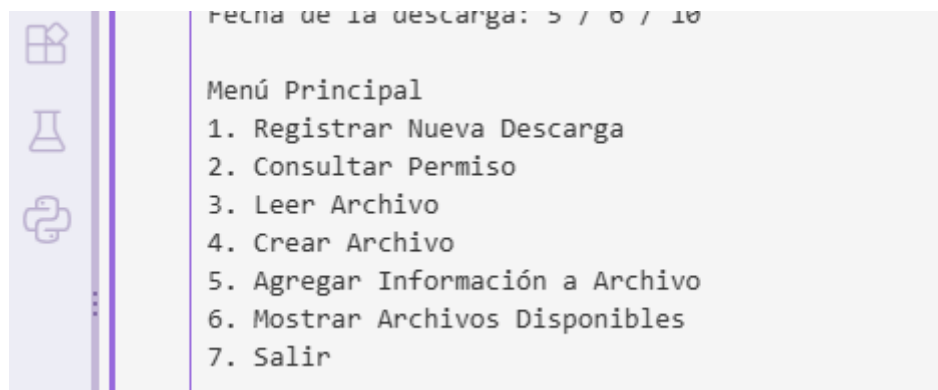
Las opciones del menú se almacenan mediante una matriz llamada `menu_matriz`. Cada fila contiene dos elementos: el número correspondiente a la opción y el nombre de la operación.

El menú actual contiene las siguientes opciones:

1. Registrar Nueva Descarga
2. Consultar Permiso
3. Leer Archivo
4. Crear Archivo
5. Agregar Información a Archivo
6. Mostrar Archivos Disponibles
7. Salir

Para recorrer la matriz se utiliza un ciclo `for`, mostrando cada una de sus filas.

El menú forma parte de un ciclo while, lo que permite que el usuario realice diferentes operaciones y posteriormente regrese al menú principal hasta seleccionar la opción Salir.



PARTE 4. ARCHIVOS DE TEXTO

El programa utiliza diferentes archivos de texto para almacenar la información de forma permanente.

Los cuatro archivos principales son:

- Permisos.txt
- Descargas.txt
- Quimicos.txt
- Recibo_Descarga.txt

Cada archivo tiene una función específica dentro del sistema.

Permisos.txt almacena información relacionada con los permisos registrados y las empresas.

Descargas.txt almacena la información correspondiente a las descargas autorizadas.

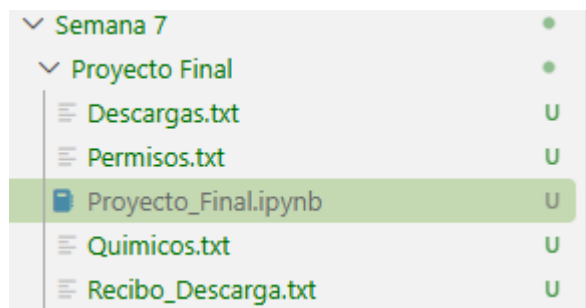
Quimicos.txt almacena las sustancias químicas identificadas junto con su concentración en ppm.

Recibo_Descarga.txt almacena un resumen o recibo de las descargas realizadas.

La función `prep_archs()` utiliza un diccionario llamado `archs_ini` que contiene como llave el nombre de cada archivo y como valor el encabezado que debe almacenarse inicialmente.

Los archivos son creados utilizando el modo "x". Este modo permite crear un archivo nuevo y genera `FileExistsError` cuando el archivo ya existe.

El programa captura esta excepción para evitar sobrescribir los archivos que ya contienen información.



PARTE 5. REGISTRO DE NUEVA DESCARGA

La opción Registrar Nueva Descarga utiliza la función registrar_descarga().

Esta función corresponde a una adaptación de la lógica desarrollada durante una etapa anterior del proyecto.

Primero se solicita el número de permiso y el tipo de permiso correspondiente a la empresa.

La variable acceso se utiliza como una variable booleana para determinar si el proceso puede continuar.

Los tipos de permiso admitidos dentro del programa son:

CONAGUA-01-001

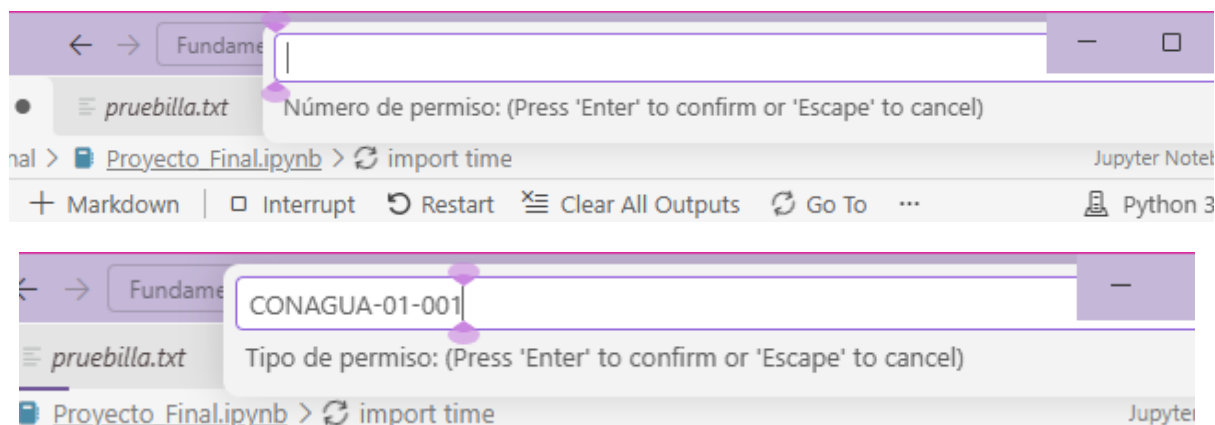
CONAGUA-01-011

Si el tipo de permiso ingresado no coincide con alguno de estos valores, el programa muestra el mensaje ACCESO DENEGADO y cambia la variable acceso a False.

Si el tipo de permiso es correcto, el sistema pregunta si el permiso se encuentra en regla.

Cuando el usuario responde “sí”, el proceso continúa. Si proporciona otra respuesta, se niega el acceso.

Cuando el permiso cumple con estas condiciones, sus datos se almacenan dentro de Permisos.txt utilizando el modo “a”, permitiendo agregar información sin eliminar registros anteriores.



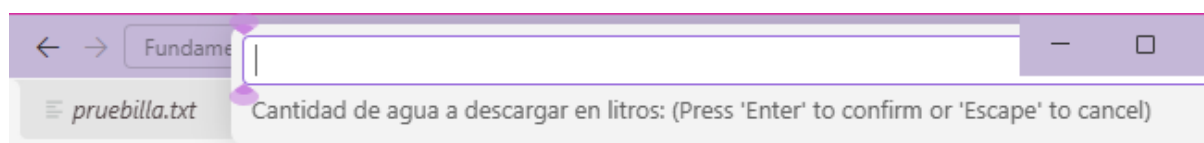
PARTE 6. REGISTRO DEL VOLUMEN DE AGUA

Después de validar el permiso, el programa solicita la cantidad de agua que se desea descargar.

El dato se captura mediante float(), permitiendo registrar cantidades con valores decimales.

La conversión se encuentra dentro de un bloque try-except para capturar ValueError si el usuario introduce un valor que no puede convertirse a número.

Si el volumen es válido, el programa continúa con la revisión de sustancias químicas.



PARTE 7. REGISTRO DE SUSTANCIAS QUÍMICAS

El sistema pregunta al usuario si el agua contiene sustancias químicas.

Cuando la respuesta es “sí”, se solicita la cantidad de sustancias presentes.

La cantidad debe ser un número entero. Para evitar errores se utiliza un bloque try-except que captura ValueError.

Posteriormente se utiliza un ciclo:

```
for contador in range(cantidad_quimicos)
```

El ciclo se repite de acuerdo con la cantidad de sustancias indicada por el usuario.

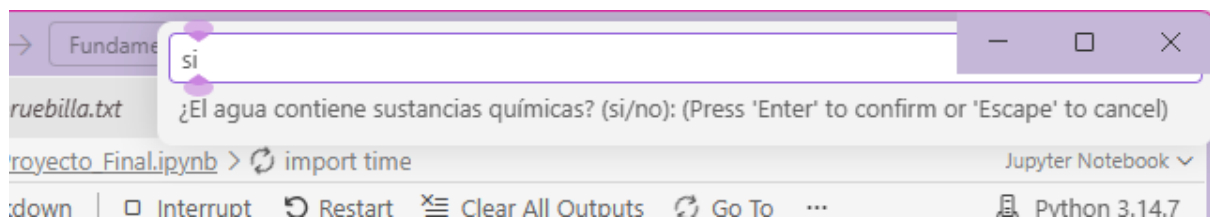
En cada repetición se solicita:

- Nombre de la sustancia.
- Concentración en ppm.

La concentración se captura mediante float().

La información de cada sustancia se almacena dentro de Quimicos.txt utilizando el modo de anexo “a”.

Cada registro incluye la fecha, empresa, nombre de la sustancia y concentración.



PARTE 8. AUTORIZACIÓN Y REGISTRO DE LA DESCARGA

Si el permiso es válido, se encuentra en regla y la operación cumple las condiciones definidas dentro del programa, la descarga es registrada en Descargas.txt.

El archivo almacena:

- Fecha.
- Empresa.
- Número de permiso.
- Tipo de permiso.
- Volumen de agua en litros.

La escritura se realiza utilizando el modo “a”, lo que permite conservar los registros existentes.

Posteriormente el programa muestra en pantalla un resumen de los datos de la empresa y el mensaje:

‘De acuerdo, tiene autorización para proseguir con su desecho.’

Adicionalmente se agrega un registro dentro de Recibo_Descarga.txt que contiene la fecha, empresa, permiso y volumen registrado.

REGISTRAR NUEVA DESCARGA

Datos de la Empresa

Empresa: pruebaco

Número de permiso: PER113

Tipo de permiso: CONAGUA-01-001

Volumen de agua: 100000.0 litros

De acuerdo, tiene autorización para proseguir con su desecho.

Semana 7 > Proyecto Final > Descargas.txt

```
1 Registro de Descargas de Agua
2 Fecha: ('05', '06', '10')
3 Descarga de prueba
4
5 Fecha: (1, 2, 3)
6 Empresa: pruebaco
7 Número de permiso: PER113
8 Tipo de permiso: CONAGUA-01-001
9 Volumen de agua: 100000.0 litros
```

10

PARTE 9. CONSULTA DE PERMISOS

El sistema incluye la opción Consultar Permiso.

La función `consultar_permiso()` solicita al usuario el número de permiso que desea buscar.

Posteriormente se abre el archivo `Permisos.txt` utilizando el modo de lectura "r".

El contenido del archivo es leído mediante `read()`.

Después se verifica si el número proporcionado por el usuario se encuentra dentro del contenido almacenado.

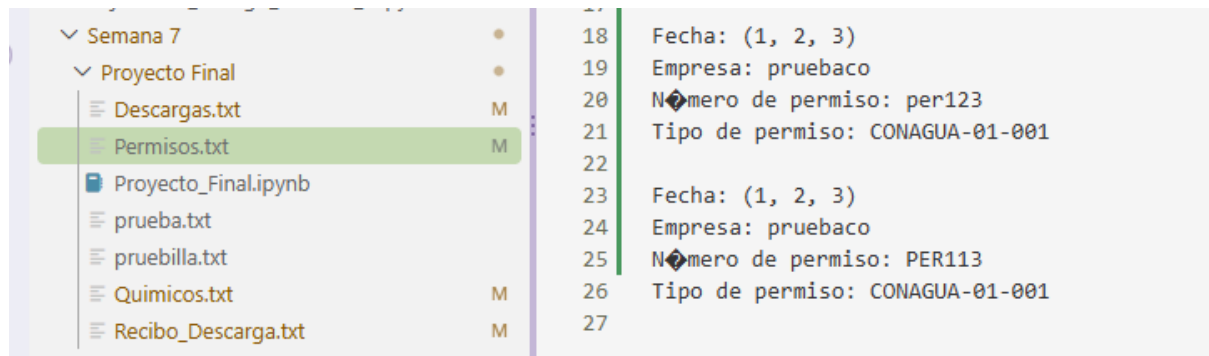
Si se encuentra una coincidencia, el sistema muestra:

'El permiso se encuentra registrado.'

En caso contrario muestra:

'El permiso no se encuentra registrado.'

También se utilizan `FileNotFoundError` y `PermissionError` para controlar posibles problemas durante la lectura del archivo.



PARTE 10. LECTURA DE ARCHIVOS

La función `leer_arch()` permite consultar los archivos disponibles dentro del sistema.

Primero se ejecuta `mostrar_archs()`, que recorre la lista `archs_disps` utilizando un ciclo `for`.

El usuario selecciona el archivo mediante el número correspondiente.

Después de realizar la selección, el archivo se abre utilizando:

```
open(nombre_arch, "r")
```

El contenido se obtiene mediante:

```
file.read()
```

Posteriormente la información se muestra en pantalla.

La selección del archivo está protegida mediante `try-except` para capturar `ValueError` cuando el usuario introduce un dato que no es numérico.

También se utilizan `FileNotFoundError` y `PermissionError` durante la apertura del archivo.

```
Archivos Disponibles
1. Permisos.txt
2. Descargas.txt
3. Quimicos.txt
4. Recibo_Descarga.txt

Contenido del Archivo
Registro de Descargas de Agua
Fecha: ('05', '06', '10')
Descarga de prueba

Fecha: (1, 2, 3)
Empresa: pruebaco
Número de permiso: PER113
Tipo de permiso: CONAGUA-01-001
Volumen de agua: 100000.0 litros
```

PARTE 11. CREACIÓN DE NUEVOS ARCHIVOS

El programa permite crear archivos adicionales mediante la opción Crear Archivo.

La función `crear_arch()` solicita el nombre del nuevo archivo y posteriormente solicita la información que se desea guardar.

Para crear y escribir el archivo se utiliza:

```
open(nomb_arch, "w")
```

Dentro del archivo se escribe primero la fecha correspondiente a la operación y posteriormente la información proporcionada por el usuario.

Después de crear correctamente el archivo, su nombre es agregado a la lista `archs_disps` para que pueda aparecer entre los archivos disponibles durante la ejecución actual.



```
1  Fecha: (1, 2, 3)
2  La conagua tiene que regular las cosas mejor
3
```

PARTE 12. AGREGAR INFORMACIÓN A UN ARCHIVO

La función `agregar_arch()` permite modificar uno de los archivos existentes sin eliminar su contenido anterior.

Primero se muestran los archivos disponibles.

Después el usuario selecciona el archivo al que desea agregar información.

El archivo se abre utilizando:

```
open(nomb_arch, "a")
```

El modo "a" permite anexar nueva información al final del archivo.

Cada modificación registra la fecha correspondiente y la nueva información proporcionada por el usuario.

Gracias a este modo, la información que ya existía dentro del archivo se conserva.

PARTE 13. MOSTRAR ARCHIVOS DISPONIBLES

La opción Mostrar Archivos Disponibles ejecuta la función `mostrar_archs()`.

Esta función utiliza un ciclo `for` junto con `range()` y `len()` para recorrer la lista `archs_disps`.

Cada archivo se presenta en pantalla acompañado de un número para facilitar su selección dentro de otras funciones del programa.

PARTE 14. CONTROL DE INACTIVIDAD

El programa cuenta con una función llamada `cont_inac()` encargada de revisar el tiempo transcurrido durante la selección de una opción del menú.

Antes de solicitar una opción se almacena el tiempo inicial mediante:

`time.time()`

Después de que el usuario proporciona una respuesta se obtiene nuevamente el tiempo y se calcula la diferencia.

La función `cont_inac()` recibe el tiempo transcurrido.

Dentro de esta función se utiliza un ciclo `for` con un rango de diez iteraciones para comprobar los diez minutos establecidos para el control de inactividad.

Cuando el tiempo alcanza 600 segundos, equivalentes a diez minutos, el sistema muestra un mensaje indicando que ha transcurrido el tiempo de inactividad.

Posteriormente pregunta:

`'¿Desea continuar? (si/no)'`

Si el usuario responde 'si', el programa continúa funcionando.

Si responde otra opción, el programa regresa a la pantalla de inicio.

PARTE 15. MANEJO DE ERRORES

Para evitar que el programa termine inesperadamente cuando ocurre un error durante la ejecución, se utilizaron diferentes bloques `try-except`.

Los principales errores controlados son los siguientes:

`ValueError`

Se utiliza cuando el programa espera un número y el usuario introduce un valor que no puede convertirse.

Este manejo se utiliza en:

- Captura de fecha.
- Selección de opciones.
- Selección de archivos.
- Volumen de agua.
- Cantidad de sustancias químicas.
- Concentración en ppm.

`FileNotFoundError`

Se utiliza durante diferentes operaciones de lectura o modificación de archivos. Permite mostrar un mensaje cuando un archivo requerido no puede ser localizado.

`PermissionError`

Se utiliza para controlar los casos en que el programa no tiene permisos suficientes para crear, leer o modificar un archivo.

`FileExistsError`

Se utiliza durante la preparación inicial de los archivos. Permite detectar que un archivo ya existe y evita que su información sea reemplazada.

El uso de estas excepciones permite que el usuario reciba un mensaje relacionado con el problema en lugar de que el programa finalice abruptamente.

PARTE 16. DEPURACIÓN MEDIANTE PDB

Para realizar la depuración técnica del programa se utilizó el módulo estándar pdb de Python.

El módulo se importa utilizando:

```
import pdb
```

Durante las pruebas se puede colocar temporalmente:

```
pdb.set_trace()
```

en una parte determinada del programa.

Esta instrucción pausa la ejecución y permite revisar el estado de diferentes variables.

Entre los comandos utilizados durante la depuración se encuentran:

```
p nombre_variable
```

Permite mostrar el valor actual de una variable.

```
n
```

Permite avanzar a la siguiente línea.

```
c
```

Permite continuar normalmente con la ejecución del programa.

Durante las pruebas se revisaron variables como opcion, fecha, empresa, acceso y cantidad_quimicos.

PARTE 17. ERRORES DETECTADOS DURANTE LA DEPURACIÓN

Durante la revisión de la versión anterior del programa se detectaron diferentes fallas de lógica que fueron corregidas para la versión final.

Error 1. Uso incorrecto de break durante el registro de sustancias

En la versión anterior existía una instrucción break dentro del ciclo utilizado para registrar sustancias químicas.

La estructura anterior provocaba que, aunque el usuario indicara que existían varias sustancias, el ciclo terminara inmediatamente después de capturar la primera.

La instrucción break fue eliminada.

Con esta corrección el ciclo:

```
for contador in range(cantidad_quimicos)
```

puede realizar todas las iteraciones correspondientes a la cantidad indicada por el usuario.

Error 2. Formato incorrecto de un tipo de permiso

Durante la revisión del código se encontró que uno de los tipos de permiso había sido escrito como:

CONAGUA -01-011

El espacio ubicado antes del primer guion ocasionaba que el valor no tuviera el mismo formato que el otro permiso.

El valor fue corregido a:

CONAGUA-01-011

Error 3. Menú sin control de selección

En una versión anterior la función mostrar_menu() únicamente mostraba las opciones en pantalla, pero no existía un ciclo principal que solicitara y procesara permanentemente la selección del usuario.

Para corregirlo se agregó un ciclo while y una estructura if, elif y else que relaciona cada opción con la función correspondiente.

Gracias a esta modificación, después de terminar una operación el sistema puede regresar nuevamente al menú.

PARTE 18. INTEGRACIÓN DEL PROGRAMA

La versión final integra todas las funciones dentro de un mismo flujo de ejecución.

El programa comienza preparando los archivos de texto mediante prep_archs().

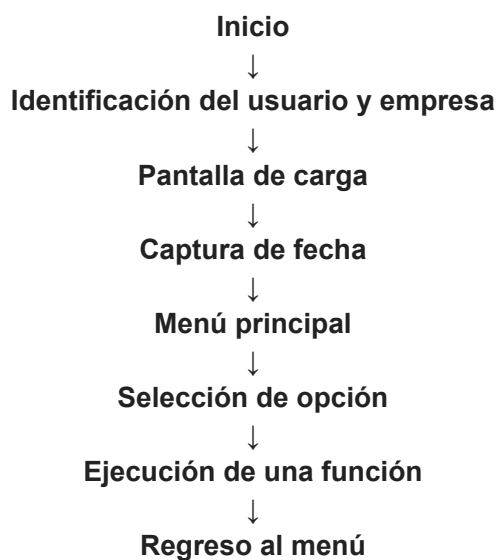
Después solicita los datos del usuario y de la empresa.

Posteriormente solicita la fecha y la almacena dentro de una tupla.

Finalmente se muestra el menú principal.

El ciclo principal puede representarse de la siguiente manera:

El ciclo continúa hasta que el usuario selecciona la opción Salir.



PARTE 21. CONTROL DE VERSIONES

Para conservar el historial de modificaciones del proyecto se utiliza Git y un repositorio público de GitHub.

Los cambios se dividen en diferentes etapas para mantener un historial comprensible del desarrollo.

feat: integrar menu principal y funciones de persistencia

fix: agregar validaciones y manejo de errores

docs: documentar proyecto y agregar evidencia de depuracion

PARTE 19. PRUEBAS DEL SISTEMA

La columna “Resultado obtenido” queda pendiente para completarse después de realizar cada prueba.

Prueba	Acción realizada	Resultado esperado	Resultado obtenido
1. Inicio del sistema	Ingresar usuario y empresa.	Mostrar bienvenida y pantalla de carga.	
2. Captura de fecha correcta	Ingresar día, mes y año numéricos.	Mostrar correctamente la fecha.	
3. Captura de fecha incorrecta	Ingresar letras.	Mostrar mensaje de error y volver a solicitar la fecha.	
4. Permiso válido	Ingresar CONAGUA-01-001 o CONAGUA-01-011.	Continuar con la validación.	
5. Permiso inválido	Ingresar otro tipo.	Mostrar ACCESO DENEGADO.	
6. Descarga sin sustancias químicas		Registrar la descarga.	
7. Descarga con sustancias químicas		Registrar las sustancias en Quimicos.txt y negar	

		la descarga según la lógica del programa.	
8. Lectura de archivo		Mostrar el contenido del archivo seleccionado.	
9. Creación de archivo		Generar un nuevo archivo utilizando "w".	
10. Anexado de información		Conservar información anterior y agregar nueva información.	
11. Consulta de permiso		Indicar si el permiso se encuentra registrado.	
12. Entrada no numérica en el menú		Capturar ValueError y mantener el programa funcionando.	
13. Salir		Finalizar el ciclo del menú.	

PARTE 20. CUMPLIMIENTO DE LOS REQUERIMIENTOS

Número	Requerimiento	Forma en que se cumple	Evidencia
1	Identificación de Usuario	La función pant_inicio() solicita el nombre o nickname del usuario.	Parte 1 y Figura 1.
2	Bienvenida Dinámica	pant_inicio() utiliza el nombre y la empresa ingresados para generar un mensaje personalizado.	Parte 1 y Figura 1.
3	Pantalla de Carga	pant_carga() utiliza un ciclo for de tres iteraciones y time.sleep(1), por lo que dura aproximadamente tres segundos.	Parte 1 y Figura 1.
4	Menú como Matriz y ciclo while	Las opciones se almacenan en menu_matriz y el programa utiliza un ciclo while para mantener activo el menú.	Parte 3 y Figura 3.
5	Control de Inactividad	Se mide el tiempo mediante time.time() y la función cont_inac() utiliza un ciclo for relacionado con los diez minutos de inactividad y solicita una respuesta si/no.	Parte 14 y Figura 20.
6	Fecha mediante Tupla	La fecha se almacena mediante Fecha = dia, mes, año y se utiliza	Parte 2 y Figura 2.

		en las operaciones con archivos.	
7	Persistencia en Archivos	El sistema utiliza como mínimo cuatro archivos de texto y operaciones de lectura, creación y anexo mediante los modos "r", "w" y "a".	Partes 4, 8, 10, 11 y 12.
8	Control de Excepciones	Se utilizan ValueError, FileNotFoundError, PermissionError y FileExistsError mediante bloques try-except.	Parte 15.
9	Depuración con PDB	Se utiliza el módulo pdb para detener la ejecución, inspeccionar variables y analizar errores de lógica.	Partes 16 y 17 y Figura 23.
10	Comentarios de Calidad	Este criterio corresponde directamente al código fuente entregado. Antes de realizar la entrega definitiva se deben verificar los comentarios descriptivos incluidos en las funciones y bloques principales del archivo Python.	

CONCLUSIÓN

El desarrollo de la versión final del Sistema de Descarga de Agua permitió integrar los diferentes temas revisados durante el curso de Fundamentos de Programación.

La primera versión del proyecto se encontraba principalmente enfocada en la validación de permisos, el volumen de agua y la presencia de sustancias químicas. Durante el desarrollo de la versión final esta lógica fue organizada mediante funciones e integrada dentro de un menú principal.

También se incorporó el uso de archivos de texto, permitiendo conservar información mediante los modos de lectura, escritura y anexo. Los bloques try-except permitieron controlar diferentes errores de ejecución y PDB fue utilizado como herramienta de apoyo durante la depuración.

Con estas modificaciones el programa integra estructuras condicionales, ciclos for y while, funciones, matrices, tuplas, archivos externos y manejo de excepciones dentro de un mismo proyecto.